

Problem - Comparison of Thread Counting Using Static and Non-Static Variables

```
import java.util.concurrent.atomic.AtomicLong;

public class ThreadCount extends Thread {
    // shared thread-safe static counter
    static AtomicLong safeStaticCounter = new AtomicLong (
        initialValue : initialValue : 0);
    // shared unsafe static counter
    static long unsafeStaticCounter = 0;
    // thread obj.'s instance counter
    long instanceCounter = 0;

    long increments;
    boolean safeMode;
    // constr.
    public ThreadCount (long increments, boolean safeMode)
    {
        this.increments = increments;
        this.safeMode = safeMode;
    }

    // Thread execution.
    @Override
    public void run () {
        for (long i = 0; i < increments; i++) {
            if (safeMode) {
                safeStaticCounter.incrementAndGet();
            }
            else {

```

it-24048

```

        unsafeStaticCounter ++ ; }
    instanceCounter ++ ; } }

    public static void main (String[] args ) {
        throws InterruptedException {
            if (args.length != 3) {
                System.out.println ("Usage : java ThreadCount <threads>
                <increments> <true/false> ");
                return ; }

```

it-24048

```

        int numberOfThreads = Integer.parseInt (arg[0]);
        long increments = Long.parseLong (args[1]);
        boolean safeMode = Boolean.parseBoolean (args[2]);

        if (numberOfThreads < 1 || increments < 0 ||
            !(args[2].equalsIgnoreCase ("true") ||
              args[2].equalsIgnoreCase ("false"))) {
            return ; }

        // Reset counters before execution
        safeStaticCounter.set (0);
        unsafeStaticCounter = 0 ;

```

```

ThreadCount[] threads = new ThreadCount [numberOfThreads]
long nonStaticTotal = 0;
// create and start for all thread
for (int i=0; i < numberOfThreads; i++) {
    threads[i] = new ThreadCount (increments, safeMode);
    threads[i].start();
    // wait for every thread to finish
    for (int i=0; i < numberOfThreads; i++) {
        threads[i].join();
    }
    // add all individual instance counter
    for (int i=0; i < numberOfThreads; i++) {
        nonStaticTotal += threads[i].instanceCounter;
    }
}
long staticTotal;
if (safeMode) {
    staticTotal = unsafeStaticCounter.get();
} else {
    staticTotal = unsafeStaticCounter;
}
long expectedCount = (long) numberOfThreads * increments;
long difference = Math.abs(staticTotal - nonStaticTotal);
double percentageDifference;

```

it-24048

```

if (nonStaticTotal == 0) {
    percentageDifference = (staticTotal == 0) ? 0.0 :
        Double.NaN; }
else {
    percentageDifference = (double) difference / nonStaticTotal * 100;
}
System.out.println("=== Thread Counter Lab ===");
System.out.println("Threads: " + numberOfThreads);
System.out.println("Increments per thread " + increment);
System.out.println("Mode: " + (safeMode ? "Thread-safe" : "Unsafe"));
System.out.println("Expected Count: " + expectedCount);
System.out.println("Static count: " + staticTotal);
System.out.println("Non-static total: " + nonStaticTotal);
System.out.println("Absolute difference: " + difference);
if (Double.isNaN(percentageDifference)) {
    System.out.printf("Percent difference: %.4f%%\n",
        percentageDifference);
}
System.out.println("=====");
}

```